ニューラルネットワークとモジュライ空間

クィバー表現論で AI を理解する —— 学部 1-2 年生のためのサーベイ論考

Technical Survey — A Study of Moduli Space Formulation of Neural Networks via Quiver Representation Theory

---

著者: Étale Cohomology
所属: （空欄）
日付: 2026 年 4 月
ライセンス: CC BY 4.0

Contents

この文書を読む前に **4**

0.1 想定読者 4

0.2 本サーベイ論考の方針 4

0.3 読み方の推奨 4

TL;DR — 本サーベイ論考の要点を 5 行で **4**

1 はじめに — AI と数学のつながりを見たい **4**

1.1 AI の中で何が起きているのか 4

1.2 2020 年の発見 5

1.3 本サーベイ論考のゴール 5

2 準備: ベクトル空間と線形写像を復習する **5**

2.1 ベクトルとは「数字の組」 5

2.2 ベクトル空間とは「数字の組を全部集めた袋」 5

2.3 線形写像とは「変換ルール」 6

2.4 線形「写像」と「変換」と「行列」は同じものか 6

2.5 ここまでのまとめ 6

3 クィバーとは何か (矢印つきのグラフ) **7**

3.1 まず見てみよう 7

3.2 正式な定義 8

3.3 名前の由来 8

3.4 記号での書き方 8

3.5 クィバーの何が面白いか (予告) 8

4 クィバー表現とは何か (グラフに数字を付けたもの) **8**

4.1 動機 — ただのクィバーだけでは情報が足りない 8

4.2 定義 9

4.3 一番簡単な例 9

4.4 もう少し大きな例 — ニューラルネットの片鱗 9

4.5 「表現」という名前について 10

4.6 なぜこれを考えるか (動機の再確認) 10

4.7 ここまでのまとめ 10

5 ニューラルネットの復習とクィバーへの翻訳 **10**

5.1 ニューラルネットをまだ知らない人のための 1 ページ入門 10

5.2 ニューラルネットの数式的な中身 11
5.3 クィバー表現との対応 11
5.4 Armenta & Jodoin の定義 12
5.5 何が嬉しいのか (メリット) 12
5.6 ここまでのまとめ 12

6 「同じ」とはどういうことか (同型の考え方) 13
6.1 日常の「同じ」と「違う」 13
6.2 クィバー表現の「同型」とは 13
6.3 簡単な例で確認 13
6.4 ニューラルネットの「同型」の例 14
6.5 技術的ハイライト: ReLU の正スケール不変性 14
6.6 同型類 (isomorphism class) 15
6.7 ここまでのまとめ 15

7 モジュライ空間 — 「同じ」を 1 点にまとめた地図 15
7.1 日常例で直感を作る 15
7.2 クィバー表現のモジュライ空間 16
7.3 具体例で確認 16
7.4 モジュライ空間の「大きさ (次元)」 16
7.5 なぜモジュライ空間が AI 研究に重要か 17
7.6 注意: 実は数学的にはかなり精巧な話 17
7.7 ここまでのまとめ 17

8 Armenta & Jodoin (2021) の中身 17
8.1 論文の基本情報 17
8.2 5 つの主要主張 17
8.3 主張 2 の詳細 — データがクィバー表現になるとはどういうことか 18
8.4 主張 3 の詳細 — モジュライ空間上のデータ雲 18
8.5 主張 4 の詳細 — 容量との一致 19
8.6 論文の構成・読み方 19
8.7 この論文の影響 19

9 その後の発展 — 6 つの論文のダイジェスト 19
9.1 論文 1: モジュライ空間を行列として書く (2022) 20
9.2 論文 2: ニューラル・テレポーテーション (2023) 20
9.3 論文 3: 解釈可能性への応用 (2024) 20
9.4 論文 4: モデルを 20-40% 圧縮する (2021) 21
9.5 論文 5: アーキテクチャの統一言語 (2022) 21

9.6 論文 6: Softmax は代数幾何から自然に出てくる (2022) 22
9.7 論文 7: CNN のクィバー版 (2023) 23
9.8 6 本のまとめ 23

10 大規模言語モデルへの応用可能性 **23**
10.1 Armenta & Jodoin のテキストへの明示的言及 23
10.2 現状は未開拓 24
10.3 理論的には何が可能か (筆者による独自考察) 24
10.4 まだ誰も書いていない研究テーマ 24

11 現状と未解決問題 **25**
11.1 現時点で確立されたこと 25
11.2 未解決の理論問題 25
11.3 未解決の工学問題 25
11.4 本分野の全体評価 25

Appendix A. 主要論文の年表 **25**

Appendix B. 用語集 **26**

Appendix C. 参考文献 **27**
11.5 C.1 本系譜の直接の研究論文 27
11.6 C.2 数学的基礎 (必要に応じて) 28
11.7 C.3 関連する機械学習理論 28

License **28**

この文書を読む前に

0.1 想定読者

- **大学 1〜2 年生レベル**: 線形代数の基礎 (ベクトル、行列の掛け算) は習ったが、抽象数学 (群、環、商空間など) は苦手
- 機械学習に興味があるが、理論は苦手
- ニューラルネットの「内側」に関心がある

0.2 本サーベイ論考の方針

このサーベイ論考は、**急がず、抽象語を避け、日常例と簡単な例を必ず先に置く**方針で書きました。難しい用語はいきなり出しません。出す前に必ず、以下の 4 段階をたどります。

1. **なぜそれが必要か** (日常例や単純な場面での動機)
2. **極めて単純な具体例** (数字が 2-3 個の小さな例)
3. **普通の具体例** (もう少し規模のある例)
4. **正式な定義** (ここで初めて抽象語が出る)

0.3 読み方の推奨

- まず本文を飛ばし飛ばし眺めて、全体像を掴む
- 図と例に注目する
- 難しい箇所はスキップして次へ進む
- 興味を持った論文だけ後で深掘りする

もし途中で難しくなったら、その場で止めて OK です。大事なのは、「**こんな視点で AI を見る研究があるんだ**」という感覚を得ることです。

TL;DR — 本サーベイ論考の要点を 5 行で

1. ニューラルネット (AI の心臓部) は、**矢印つきの図**に数字の情報を貼り付けたものとして、数学的にピッタリ書ける。
2. AI を訓練するとき、各入力データは「**モジュライ空間**」**という地図上の一点**として表現される。
3. この地図の「広さ」は、AI の学習能力と正確に一致する。
4. この視点から、**AI の圧縮、学習の高速化、内部の解釈**といった具体的応用が出てきた。
5. この分野は 2020 年に始まったばかりで、まだ誰も書いていない研究がたくさん残っている。

---

1 はじめに — AI と数学のつながりを見たい

1.1 AI の中で何が起きているのか

ChatGPT や画像生成 AI は、内部ではニューラルネットワークという計算機が動いています。これは数百万〜数兆個の数字 (パラメータ、重み) を持ち、訓練でこの数字を少しずつ調整することで問題を解けるようになります。

しかし、「**なぜ訓練するとうまくいくのか**」は実はあまり理解されていません。大勢の AI 研究者は実験を繰り返して経験的に改良していますが、それとは別に「AI の中で起きていることを数学の言葉で正確に書き下したい」という流れがあります。

1.2 2020 年の発見

2020 年、カナダの若い数学者 Marco Armenta と機械学習研究者 Pierre-Marc Jodoin は、1 本の論文でこう主張しました。

> ニューラルネットワークは、**クィバー表現 (quiver representation)** という数学の対象として、「そっくりそのまま」書き直せる。

ここで **クィバー**とは英語で「矢筒 (矢を入れる筒)」のこと。数学用語としては 「**矢印つきのグラフ**」の意味で使われます。矢筒にたくさんの矢が詰まっているイメージですね。

この論文がきっかけで、以降 4-5 年の間に「クィバー表現論で AI を理解する」研究が増えてきました。本サーベイ論考はその流れを紹介します。

1.3 本サーベイ論考のゴール

- **クィバー**とは何かを、図と例で理解する
- **クィバー表現**とは何かを、数字のついた小さな例で理解する
- **モジュライ空間**という「似たものをまとめた地図」を理解する
- これらを使って、Armenta & Jodoin が何を発見したかを理解する
- 後続の 6 本の論文の概要を掴む

---

2 準備: ベクトル空間と線形写像を復習する

この章の役割: クィバー表現を学ぶ前に、道具である「ベクトル空間」と「線形写像」を思い出す。学部 1 年の線形代数で習った内容ですが、「**何の役に立つのか**」**に焦点を置いて**復習します。

2.1 ベクトルとは「数字の組」

ベクトルの正体は、「**数字を一列に並べたもの**」です。

例: - $(3, 5)$ は 2 次元ベクトル (2 つの数字の組) - $(1.2, -0.7, 3.5)$ は 3 次元ベクトル - $(0.1, 0.3, -0.2, 0.8, \ldots)$ のように 784 個並べれば 784 次元ベクトル

日常例: 天気予報の情報 (気温、湿度、気圧) を $(25.3, 60, 1013)$ というベクトルで表す、身体測定の結果 (身長、体重、視力) を $(170, 65, 1.2)$ と表す。「**ある物事の特徴を数字の組にまとめたもの**」と思うと分かりやすいです。

2.2 ベクトル空間とは「数字の組を全部集めた袋」

「**2 次元ベクトル**」**を全部集めた袋**を考えましょう。中には $(3, 5), (-1, 2), (100, 0.5), \ldots$ などあらゆる 2 次元ベクトルが入っています。この袋のことを 「**2 次元のベクトル空間**」と呼び、数学の記号では $\mathbb{R}^2$ と書きます。

- $\mathbb{R}$ は「実数 (Real numbers) 全体」の記号。1.5 も -3 も π も入っている。

- $\mathbb{R}^2$ は「実数を 2 つ並べたもの全体」。つまり 2 次元ベクトルの袋。
- $\mathbb{R}^3$ は 3 次元ベクトル全体。
- $\mathbb{R}^{784}$ は 784 次元ベクトル全体。

ベクトル空間の正体は、単に「ある次元の数字の組、全部集めた袋」です。難しく考えなくて大丈夫です。

なぜ「空間」と呼ぶか: 2 次元ベクトル (x, y) は平面上の点と同じです。だからベクトルの袋は平面全体と見做せます。これが「空間」という名前の由来です。3 次元も同様に立体の空間に対応します。4 次元以上は想像しづらいですが、数学的には同じ扱いです。

2.3 線形写像とは「変換ルール」

線形写像とは、ベクトル空間からベクトル空間への「変換ルール」のことです。重要なのは、この「ルール」が **行列で書ける**ということ。

例: $\mathbb{R}^2$ から $\mathbb{R}^2$ への変換ルールの 1 つは、たとえば

$$\text{ルール L: 入力 } (x, y) \text{ に対して、出力 } (2x + y, 3y) \text{ を返す}$$

これは行列で書くと以下の通り:

行列の式で書くと、出力 $(2x + y, 3y)$ は、行列 `[[2, 1], [0, 3]]` を列ベクトル $(x, y)^T$ に掛けた結果です。ここで `[[2, 1], [0, 3]]` は「1 行目が 2 と 1、2 行目が 0 と 3」という 2×2 行列を表しています。

ここで行列 `2, 1, 0, 3` が、ルール L を表す「数字の組」です。

一般的な事実: 有限次元のベクトル空間の間の線形写像は、常に行列 1 つで書けます。

- $\mathbb{R}^3 \to \mathbb{R}^2$ なら、2×3 行列 1 つ
- $\mathbb{R}^5 \to \mathbb{R}^7$ なら、7×5 行列 1 つ
- 一般に $\mathbb{R}^m \to \mathbb{R}^n$ なら、$n \times m$ 行列 1 つ

日常例: - 為替レートの表は線形写像: ドル → 円の変換 (1 ドル = 150 円) は、$\mathbb{R} \to \mathbb{R}$ の線形写像で、行列は 1 つの数 150 - カラー画像の色変換 (RGB → グレー): $\mathbb{R}^3 \to \mathbb{R}$ の線形写像

2.4 線形「写像」と「変換」と「行列」は同じものか

はい、本サーベイ論考では事実上同じものとして扱います。

- **線形写像** (mathematicians'jargon): 抽象的な呼び方
- **線形変換**: 同じ空間から同じ空間への線形写像
- **行列**: 有限次元の場合の具体的な表現

本稿を読むうえでは「**線形写像 ≈ 行列**」と思ってもほとんど問題ありません。

2.5 ここまでのまとめ

3 つの言葉を覚えました。

用語	正体	日常イメージ
ベクトル	数字の組	「身長、体重、視力」のような複数の数値の記録
ベクトル空間	ある次元のベクトル全部の袋	「2 次元平面」「3 次元空間」のような場所

用語	正体	日常イメージ
線形写像	ベクトル空間の間の変換ルール	「為替レート」「色変換」のような数値変換

これらが 2 章以降で何度も使われます。困ったらここに戻ってきてください。

3　クィバーとは何か (矢印つきのグラフ)

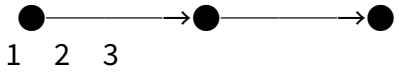
この章の役割: 新出語は 1 つだけ —「クィバー」。それを図でたっぷり見て、イメージに落とし込む。

3.1　まず見てみよう

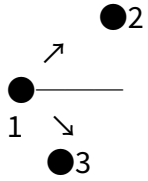
以下はすべて「クィバー」です。

クィバー例 A (点 2 つ、矢印 1 本):

クィバー例 B (点 3 つ、矢印 2 本が直列):



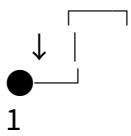
クィバー例 C (点 3 つ、枝分かれ):



クィバー例 D (点 2 つ、矢印 2 本が並列):

(2 本の矢印を ═══→ で表現。実際は —α→ と —β→ と区別します)

クィバー例 E (点 1 つ、ループ):



点 1 から自分自身に戻る矢印。

3.2 正式な定義

以上のイメージを定義にまとめます。

定義: クィバーとは、以下の 2 つの情報からなる図形のこと。

1. **頂点** (点) の有限集合
2. **矢印** (有向辺) の有限集合。各矢印は「出発点の頂点」と「終点の頂点」を持つ

許されること: - 同じ 2 点を結ぶ矢印が複数あっても OK (例 D) - 自分自身に戻るループがあっても OK (例 E) - どの方向に描いてもよい

要するに: 矢印を自由に引いた図を全部クィバーと呼びます。

3.3 名前の由来

「クィバー (quiver)」は英語で「矢筒 (矢を入れる筒)」のこと。図に矢印が詰まっているから、こう呼ばれるようになりました。ドイツ人数学者 Peter Gabriel が 1970 年代にこの用語を定着させました。

3.4 記号での書き方

数学者はクィバーを `Q` という記号で表します。例 A のクィバーは

$$Q : 1 \to 2$$

例 B は

$$Q : 1 \to 2 \to 3$$

と書きます。この記法は本稿でも使います。

3.5 クィバーの何が面白いか (予告)

一見、クィバーは「矢印つきのグラフ」というだけの単純な対象です。なぜこれが AI の数学化に使えるのでしょうか。

ヒント: **ニューラルネットの構造も、点 (ニューロン) と矢印 (接続) からできている**。4 章で詳述します。

---

4 クィバー表現とは何か (グラフに数字を付けたもの)

この章の役割: 新出語は「クィバー表現」。これがニューラルネットの数学化で最重要の概念です。急がずじっくり。

4.1 動機 — ただのクィバーだけでは情報が足りない

クィバーは「点と矢印」だけの骨組みで、数値的な情報を持っていません。ニューラルネットには「重み」という数字の情報があるので、クィバーに**数字を貼り付ける**必要があります。

貼り付け方は 2 つ。

- **各点に、何か数字 (ベクトル) を置く**
- **各矢印に、何か変換ルール (行列) を置く**

これで骨組みだったクィバーに数値情報が乗ります。これが「クィバー表現」の正体です。

4.2 定義

定義: クィバー Q の **表現**とは、以下の 2 つの情報からなる。

1. **各頂点** i に、**ベクトル空間** V_i を 1 つ割り当てる (数字の袋)
2. **各矢印** (出発点 i、終点 j) に、**線形写像** $W : V_i \to V_j$ を 1 つ割り当てる (変換ルール、すなわち行列)

要するに: クィバーの各点に「ベクトル空間」、各矢印に「行列」を置いたもの。これが「クィバー表現」です。

4.3 一番簡単な例

例 A のクィバー: $Q : 1 \to 2$

このクィバーの表現を 1 つ作りましょう。

- 頂点 1 に $V_1 = \mathbb{R}^2$ を置く (2 次元ベクトルの袋)
- 頂点 2 に $V_2 = \mathbb{R}^3$ を置く (3 次元ベクトルの袋)
- 矢印に、$V_1 \to V_2$ の線形写像、つまり 3×2 行列 1 つを置く。たとえば

```
W = [[1,  2],
     [0,  1],
     [-1, 3]]
```

(3 行 2 列の行列)

これでクィバー $1 \to 2$ の「表現」が 1 つできました。

別の表現を作る: 行列を別のものに置き換えれば、別の表現になります。

```
W' = [[5, -1],
      [2,  0],
      [1,  1]]
```

これは別の表現。同じクィバー $1 \to 2$ 上で、表現は行列の選び方で無数に作れます。

4.4 もう少し大きな例 — ニューラルネットの片鱗

例 B のクィバー: $Q : 1 \to 2 \to 3$

3 層ニューラルネットのような骨組みです。次のような表現を作ります。

- 頂点 1 に $V_1 = \mathbb{R}^{784}$ (MNIST の 28×28 画像を並べたベクトル)
- 頂点 2 に $V_2 = \mathbb{R}^{128}$ (隠れ層、128 個のニューロン)
- 頂点 3 に $V_3 = \mathbb{R}^{10}$ (10 クラスの分類出力)
- 矢印 1→2 に、128×784 の行列 W_1
- 矢印 2→3 に、10×128 の行列 W_2

お気づきでしょうか: これは MNIST 手書き数字分類の **2 層ニューラルネットの重みそのもの**です。

ただし、まだ活性化関数 (ReLU, sigmoid 等の非線形関数) が入っていません。それは 4 章で扱います。

4.5 「表現」という名前について

数学的に言うと、クィバー「表現」は、クィバーという抽象的な骨組みを、具体的なベクトル空間と行列で「表現 (具体化)」したもの、という意味です。「representation」という英単語を訳したもので、「代表」「具現化」の意味合いを持ちます。

ゴリ押し理解法: 「クィバー表現 = 矢印つき図に、各点で数字の袋と、各矢印で行列を割り当てたもの」と覚えれば、本サーベイ論考では十分です。

4.6 なぜこれを考えるか (動機の再確認)

ここまで長かったので、動機を再確認しましょう。

私たちは **ニューラルネットを数学的に書き下したい**のでした。そのため:

- ニューラルネットの **層のニューロン** = クィバーの **頂点**
- ニューラルネットの **接続** = クィバーの **矢印**
- ニューラルネットの **重み行列** = クィバー表現の **線形写像 (行列)**

という対応が得られます。次の 4 章で、これを正確に書きます。

4.7 ここまでのまとめ

段階	中身
クィバー	矢印つきのグラフ (点と矢印だけ)
クィバー表現	各点にベクトル空間、各矢印に行列を割り当てたクィバー

この 2 段階がわかれば、次章以降もついていけます。

---

5 ニューラルネットの復習とクィバーへの翻訳

この章の役割: ニューラルネットを一度も見たことがない読者のためにも、ゼロから丁寧に説明する。その上で、クィバー表現としてどう書けるかを正確に示す。

5.1 ニューラルネットをまだ知らない人のための 1 ページ入門

ニューラルネットとは何か:

ニューラルネット (neural network, 以下 NN) は、人間の脳の神経細胞 (ニューロン) の働きを模した計算機です。もっと身近な言い方をすると:

> **入力を受け取り、たくさんの数字の掛け算と足し算を繰り返して、最後に何らかの出力を返す関数**

です。

具体的な例: 手書き数字 (0〜9) の画像を見て「これは何の数字か」を当てる機械を作りたいとします。

1. **入力**: 28×28 ピクセルの画像 → 784 個の数値を持つベクトル
2. **処理**: 内部で、この 784 個の数字を「重み」と呼ばれる別の数字たちと掛け算・足し算して変換する。これを何段階も繰り返す
3. **出力**: 10 個の数字 (各クラス 0〜9 への確信度)。最大値が予測する数字

なぜうまくいくのか:

NN には「重み」と呼ばれる大量のパラメータ (数百万〜数兆個) があります。これを **訓練**と呼ぶプロセスで少しずつ調整することで、正しい答えを出すように学習します。訓練には大量のデータ (入力と正解のペア) と計算資源が必要です。

層構造:

NN は「層 (layer)」という単位で整理されます。

- **入力層**: 入力ベクトルそのもの
- **隠れ層**: 中間の計算。複数あることが多い
- **出力層**: 最終的な答え

各層には「ニューロン」と呼ばれる小さな計算単位があり、隣の層のニューロンと接続されています。

これ以上の詳細は本サーベイ論考には不要です。上記のイメージを持っていれば十分です。

5.2 ニューラルネットの数式的な中身

NN の各層での計算を正確に書きます。

1. 入力ベクトル x を受け取る (例: 784 次元ベクトル)
2. 重み行列 W_1 を掛けて、**活性化関数** σ を適用する: $h_1 = \sigma(W_1 x)$
3. 次の重み行列 W_2 を掛けて、また活性化: $h_2 = \sigma(W_2 h_1)$
4. 以下、同様に層を繰り返し、最終出力 y を得る

活性化関数 σ とは、非線形な関数です。代表例:

- **ReLU**: $\sigma(x) = \max(0, x)$ (負の数は 0 にする関数)
- **シグモイド**: $\sigma(x) = 1/(1 + e^{-x})$ (0 から 1 に滑らかに変化)
- **tanh**: $\sigma(x) = \tanh(x)$ (-1 から 1 に滑らかに変化)

なぜ活性化関数が必要か: 重み行列の掛け算だけを何層重ねても、結局 1 つの行列と等価な計算にしかなりません。非線形関数を間に挟むことで、初めて「曲線的な」関数を表現できるようになります。これが深層学習の核心です。

5.3 クィバー表現との対応

対応表:

ニューラルネット用語	クィバー表現論での対応
層 (入力層・隠れ層・出力層)	クィバーの頂点のまとまり
ニューロン (層内の素子)	クィバーの 1 頂点
層と層の間の全結合	クィバーの 1 本の矢印 (重み行列 1 枚を担う)
重み行列	矢印に割り当てる線形写像
活性化関数	各頂点に追加する非線形関数

矢印の数え方について: 同じ対応関係でも、2 つの流儀があります。

- **流儀 A (Armenta & Jodoin の主流)**: 層と層の間を **1 本の矢印**で表し、そこに重み行列全体を線形写像として乗せる。たとえば入力 3 次元・隠れ 4 ニューロンなら、矢印 1 本に 4×3 行列を割り当てる
- **流儀 B (細粒度表現)**: 個々のニューロン同士の接続ごとに矢印を引き、各矢印に重みの 1 成分 (スカラー) を載せる。同じ例なら矢印は $3 \times 4 = 12$ 本

本サーベイ論考では **流儀 A (層間 1 矢印)** を採用します。Armenta & Jodoin (2021) で network quiver を定式化する際にもこの流儀が主流です。

具体例 (流儀 A): 入力次元 3、隠れ層 4 ニューロン、出力 2 の 2 層 MLP。

- クィバーは、入力層を表す頂点・隠れ層を表す頂点・出力層を表す頂点 (あるいはニューロン数に応じた頂点) を持つ
- 入力層 → 隠れ層の矢印: **1 本** (そこに 4×3 重み行列を割り当てる)
- 隠れ層 → 出力層の矢印: **1 本** (そこに 2×4 重み行列を割り当てる)
- 各隠れニューロンに ReLU を付加

これで、ニューラルネットの骨組み部分がクィバー、数字部分がクィバー表現、非線形部分が活性化関数として分離整理されました。

5.4 Armenta & Jodoin の定義

正式には、Armenta & Jodoin (2021) は以下のように定義しました。

定義 (ニューラルネット = クィバー表現 + 活性化関数):

ニューラルネットとは、次の 2 つの組 (W, f) である。

- W: あるクィバー (ループを除いた部分) の表現 (重み)
- $f = (f_v)_v$: 各頂点 v に割り当てられた活性化関数

これは近似ではなく、現実のニューラルネットの動作と **ぴったり一致**すると著者らは証明しました。

5.5 何が嬉しいのか (メリット)

なぜわざわざクィバーの言葉で書き直すのか。3 つの理由があります。

メリット 1: 全てのアーキテクチャが統一的に扱える

全結合層、畳み込み、残差接続、バッチ正規化、プーリング、さらには RNN、Transformer も、すべて **クィバーの構造の違い**として書けます。これは強力な統一言語です。

メリット 2: データとネットワークを対等に扱える (Armenta & Jodoin の最大の発見)

従来、「ニューラルネットの重み」と「入力データ」は別のものとして扱われていました。しかしこの枠組みでは、**入力データも別のクィバー表現として書ける**。これが 7 章で詳述する主発見です。

メリット 3: 数学の道具が使える

「クィバー表現」の研究は 1970 年代から蓄積されてきた、代数学の中心分野の 1 つです。50 年分の定理・手法が使えるようになります。

5.6 ここまでのまとめ

- ニューラルネット = クィバー + 表現 (重み) + 各頂点の活性化関数
- この見方をすると、アーキテクチャの多様性が 1 つの言語で書ける

次の 5 章では、クィバー表現の「同じ」を議論します。ここが次の重要概念です。

---

6 「同じ」とはどういうことか (同型の考え方)

この章の役割: 「2 つのクィバー表現が同型である」とはどういうことかを、日常例から始めて理解する。6 章のモジュライ空間の準備。

6.1 日常の「同じ」と「違う」

まず、数学を離れた「同じ」の例を考えましょう。

例 1: A さん「私のクラスには 30 人いる」、B さん「私のクラスには 3 ダースの人がいる」。

この 2 人は違うことを言っているか? 数字は違うが、「3 ダース = 36 人」と「30 人」は違うから、これは **違う状況**。

例 2: A さん「5 人の椅子がある」、B さん「椅子が 5 個ある」。

これは **同じ状況**を別の表現で言っている。

例 3: A さん「私の家族は 4 人家族」、B さん「私の家には 4 人住んでいる」。

厳密には違うが、多くの場合「同じ意味」として扱える。

数学でも同じことが起きます。「**表面的な表現は違うが、本質的に同じ情報を持っている**」状況を、数学では **同型 (isomorphism)** と呼びます。

6.2 クィバー表現の「同型」とは

動機: ニューラルネットでは、**ニューロンの並べ替えや、層内部の基底の取り替え**で、表面上は違う重みになっても、同じ関数を計算している場合があります。

たとえば、隠れ層に 3 つのニューロンがあって、それらをシャッフルして順序を変えても、入力と出力の関係は同じです。ただし、重み行列の書き方は変わります。

つまり: 2 つのクィバー表現が「基底の取り替えだけで一方から他方に変換できる」とき、それらは **同型**と呼びます。

6.3 簡単な例で確認

クィバー $Q : 1 \to 2$、両点とも $V_1 = V_2 = \mathbb{R}^2$ とします。

表現 A: 矢印に対応する行列を、対角行列 `diag(2, 3)` (対角線上に 2 と 3 が並び、他は 0 の 2×2 行列) とする。

表現 B: 矢印に対応する行列を、対角行列 `diag(3, 2)` とする。

A と B は同型か?

2 つの点の座標軸を交換する変換を考えます。点 1 の 1 番目の成分と 2 番目の成分を入れ替え、点 2 でも同じ入れ替えを行う。すると A の行列が B の行列に変換されます。

つまり、座標軸の取り方 (基底) を変えるだけで A と B は一致する。これが「同型」の定義です。

6.4 ニューラルネットの「同型」の例

具体例で見ましょう。3 層 MLP で、隠れ層の 2 個のニューロンを順番入れ替えたものは、入れ替え前と同じ関数を計算しますが、重み行列の各行や各列が入れ替わっています。この 2 つの表現は「同型」。

6.5 技術的ハイライト: ReLU の正スケール不変性

もう 1 つ、より繊細な同型変換があります。**各隠れニューロンの出力を定数倍して、次の層の入力を同じ定数で割る**という操作です。

なぜこれが同型になるのか (ReLU の場合):

ReLU 関数 $\mathrm{ReLU}(x) = \max(0, x)$ は、**正スケール不変性**という重要な性質を持ちます。つまり、任意の正の実数 $\alpha > 0$ について:

$$\mathrm{ReLU}(\alpha x) = \alpha \mathrm{ReLU}(x)$$

これが成り立つ。直感的には、「ReLU は正の入力に対して恒等関数、負の入力に対して 0」なので、入力を正の定数倍しても出力は同じ比率で変わるからです。

ここから何が言えるか:

たとえば、ある隠れニューロンに入る重みを全て α 倍し、そのニューロンから出る重みを全て $1/\alpha$ で割ったとします。すると:

- 入ってくる信号は α 倍になる
- ReLU を通ると、出力も α 倍になる (正スケール不変性)
- 出ていく重みで $1/\alpha$ 倍されるので、結局次の層への信号は変わらない

つまり、**ネットワーク全体が計算する関数はまったく同じ**ままで、重みの値だけが変わっています。

なぜこれが重要か (技術的意義):

- **バッチ正規化 (Batch Normalization) の理論的基盤**: バッチ正規化は、ニューロン出力を平均 0・分散 1 に正規化した後にスケールとバイアスを再学習する技法。この「スケールとバイアス」の調整は、まさに正スケール不変性が許す同型変換の一部です
- **深層学習の汎化理論**: ReLU NN のパラメータ空間には、正スケール変換による連続的な同型自由度が無数に存在するため、「真に異なるネットワーク」を数える際には注意が必要。この議論は Path-SGD (Neyshabur et al. 2015) や Dinh et al. (2017) の Sharp Minima 論文で展開されている
- **勾配消失問題との関連**: 同型変換で重みのスケールを変えられるため、勾配の大きさを人為的に調整できる

他の活性化関数との違い:

この性質は ReLU の特権的な性質で、**sigmoid や tanh ではこの正スケール不変性は成立しません**。

- シグモイド $\sigma(x) = 1/(1 + e^{-x})$: 入力を α 倍すると $1/(1 + e^{-\alpha x})$ となり、一般に $\alpha\sigma(x)$ とは全く異なる値
- tanh: 同様に、正スケール不変性を持たない

したがって、**ReLU NN だけが広い同型変換の自由度を持つ**。この事実が、後に Ganev & Walters (2022) が radial activation を一般化する動機にもなっています。

まとめ: ReLU の正スケール不変性は、単なる技術的観察ではなく、**深層学習のパラメータ空間の対称性全体を特徴づける**基本的な性質です。これが 6 章のモジュライ空間 (同型で割った空間) の

構造の豊かさの源泉になります。

6.6 同型類 (isomorphism class)

「互いに同型なクィバー表現を 1 つにまとめたもの」を **同型類**と呼びます。

日常例:「同じ曜日の日付」を 1 つにまとめる。2024 年 1 月 1 日 (月)、2024 年 1 月 8 日 (月)、2024 年 1 月 15 日 (月) …これらは全部「月曜日」という同型類に属します。

クィバー表現の場合: 2 つの同型な表現 A と B は、「同じ同型類」に属します。

同型類をうまく使うと、冗長性を除いた「本質的な分類」ができます。これが次章のモジュライ空間の準備です。

6.7 ここまでのまとめ

- **同型 (isomorphism)**: 2 つのクィバー表現が、基底の取り替えだけで一致する関係
- **ReLU の正スケール不変性**: 同型変換の一大源泉。バッチ正規化等の理論根拠
- **同型類 (isomorphism class)**: 互いに同型な表現を 1 つにまとめた集合

ニューラルネットの文脈では、「同じ関数を計算する重みの取り方の冗長性」を表す概念です。

---

7 モジュライ空間 — 「同じ」を 1 点にまとめた地図

この章の役割: いよいよ本サーベイ論考の核となる概念「モジュライ空間」。じっくり、日常例から。

用語についての注記: 本サーベイ論考では平易さを優先して「モジュライ空間」と呼びますが、より厳密には **商空間 (quotient space)** や **オービフォールド (orbifold)** (特異点を持つ商空間) として定式化するのが正確です。代数幾何学的に厳密な議論や、圏論的な扱いに興味がある読者は、本サーベイ論考とは別の論考 (たとえば『圏論と機械学習・ディープラーニングモデル』のような圏論・代数幾何学の専門サーベイ論考) を参照してください。本サーベイ論考単独では、「同型な表現を 1 つの点にまとめた空間」という直感的理解で十分です。

7.1 日常例で直感を作る

日常例 1: 曜日のモジュライ空間

- すべての日付を集めた空間を考える。2020 年 1 月 1 日、2020 年 1 月 2 日、…、無限に続く
- 「同じ曜日の日付は同一視する」というルールを入れる
- すると、分類の結果、**月、火、水、木、金、土、日**の 7 つのクラスにまとまる
- 「7 つのクラスからなる空間」が、**日付空間を曜日同値で割ったモジュライ空間**

これが「モジュライ空間」という考え方の一番簡単な例です。「**似たものをまとめて、別物として分類した結果の空間**」です。

日常例 2: 顔写真のモジュライ空間

- すべての顔写真を集めた空間を考える
- 「同じ人の写真は同一視する」というルールを入れる
- すると、分類の結果、**人物ごとのクラス**にまとまる
- 「個々の人物」を点とする空間が、顔写真のモジュライ空間

日常例 3: 分数のモジュライ空間

- すべての分数 (例: $1/2, 2/4, 3/6, 5/10 \cdots$) を集めた空間を考える
- 「値が同じ分数は同一視する」というルールを入れる
- すると、各有理数が 1 つの点になる。$1/2, 2/4, 3/6, \ldots$ はすべて同じ点 0.5
- この「有理数全体」が、分数のモジュライ空間

これらに共通するのは、「**何らかの等しい条件で分類し、代表者 (クラス) を点とする**」という構造です。

7.2 クィバー表現のモジュライ空間

定義 (直感版): クィバー Q の **モジュライ空間** $\mathcal{M}$ とは、

> **Q の表現全体を、同型類で分類した結果の空間**

のことです。数式で書くと:

$$\mathcal{M} = \{\text{All representations}\}/\{\text{Basis-change equivalence}\}$$

ここで「/」の記号は「同値関係で分類する」という意味。**表現の空間を、同型という等しさで割った結果**を表します。

7.3 具体例で確認

クィバー $Q : 1 \to 2$ を考えます。

- 両点の次元を固定する: $V_1 = \mathbb{R}^2, V_2 = \mathbb{R}^2$
- この場合、矢印に対応する行列は 2×2 行列
- 2×2 行列には 4 個の数字があるから、**表現全体は 4 次元の空間** $\mathbb{R}^{2\times 2}$

モジュライ空間を作るには、「基底変換で同じと見做す」という同値関係で割ります。

基底変換は: - 点 1 で任意の可逆 2×2 行列 P (4 つの数字で、ある条件を満たす) で変換 - 点 2 で任意の可逆 2×2 行列 Q で変換

同型を使って、任意の行列を **標準形 (Jordan 形)** に変換できるので、モジュライ空間の点は「行列の不変量 (トレース、行列式、固有値など)」で特徴づけられます。

結果: モジュライ空間の大きさは、表現全体の 4 次元から、基底変換の自由度の分を引いた値になります。

7.4 モジュライ空間の「大きさ (次元)」

モジュライ空間が大きいか小さいかは、「次元」という数値で測れます。おおまかには:

モジュライ空間の次元 ≈ 表現全体の次元 − 基底変換の自由度の次元

日常例で確認:

- 曜日のモジュライ空間: 日付空間は無限にあるが、曜日同値で割ると **7 クラス = 有限 (0 次元)**
- 顔写真のモジュライ空間: 写真全体は多次元だが、人物で分類すると **人物の数だけのクラス**

クィバーの場合も同様に、行列全体から基底変換を除いた「本質的な次元」がモジュライ空間の次元です。

7.5 なぜモジュライ空間が AI 研究に重要か

ニューラルネットでは、「同じ関数を計算する重みの取り方」が無数にあります。これらの冗長性を除いた「**本質的な重みの空間**」こそが、AI の性質を決める量です。

モジュライ空間 = ニューラルネットの本質的なパラメータ空間

これが、本サーベイ論考でモジュライ空間が最重要の概念である理由です。

7.6 注意: 実は数学的にはかなり精巧な話

以上の説明は「直感版」です。本当に厳密にモジュライ空間を作るには:

- **Geometric Invariant Theory (GIT, 幾何学的不変式論)**: Mumford 由来の 1960 年代の代数幾何学の手法
- **King の安定性条件 (1994)**: クィバー表現で特に有効な技法

など、大学院レベルの代数幾何を使います。本サーベイ論考では、「**同型な表現を 1 点にまとめた空間**」という直感だけで十分です。厳密な構成は、興味があれば Appendix C の Kirillov (2016) の教科書を参照してください。

7.7 ここまでのまとめ

- **モジュライ空間**: 同型類を 1 つの点とする空間 (似たものをまとめた地図)
- **次元**: 表現全体の次元から、基底変換の自由度を引いたもの
- **ニューラルネットとの関係**: 冗長性を除いた「本質的な重みの空間」

ここまで理解できれば、7 章で本分野の出発点である論文の内容がスッと入ります。

---

8 Armenta & Jodoin (2021) の中身

この章の役割: 本分野の起点である論文の主張を、細部の数式に惑わされずに理解する。

8.1 論文の基本情報

- **タイトル**: The Representation Theory of Neural Networks
- **著者**: Marco Antonio Armenta, Pierre-Marc Jodoin (カナダ・シェルブルック大学)
- **arXiv 初版**: 2020 年 7 月
- **MDPI Mathematics 掲載**: 2021 年 12 月 (Mathematics 9(24) 3216)
- **本文 (Open Access)**: https://www.mdpi.com/2227-7390/9/24/3216

8.2 5 つの主要主張

論文が証明したことは、大きく以下の 5 つです。

主張 1 — ニューラルネットは厳密にクィバー表現である

「近似」「比喩」ではなく、1:1 に対応します。どんなアーキテクチャ (CNN、ResNet、Transformer) もこの枠組みで書けます。

主張 2 — 入力データもクィバー表現として書ける

これが一番衝撃的な発見です。「データ x をネットワークに通した結果、何が起きるか」を、**データ自体の別のクィバー表現**として符号化できることを示しました。

主張 3 — データはモジュライ空間上の点に写像される

入力データ集合 $\{x_1, x_2, \ldots\}$ は、それぞれ **モジュライ空間 $\mathcal{M}$ 上の点**として表現されます。これによりデータが幾何学的に見える化されます。

主張 4 — モジュライ空間の次元は、ニューラルネットの「容量」と一致する

「容量」とは、別の研究者 (Zheng ら 2019) が **統計的・経験的な動機**で定義した、ニューラルネットの表現能力の尺度。これが **代数幾何学的な量であるモジュライ空間の次元**と一致する。全く違う動機の 2 つの量が一致するのは、深い構造の示唆です。

主張 5 — Manifold hypothesis の数学的実装

「高次元データは低次元の曲がった面 (多様体) に乗っている」という経験則 (manifold hypothesis) を、**モジュライ空間として具体的に構成**した。

8.3 主張 2 の詳細 — データがクィバー表現になるとはどういうことか

主張 2 が一番分かりにくいので、じっくり解説します。

設定: 既に訓練済みのニューラルネット (重み W、活性化関数 f) があります。ある入力 x をネットワークに通します。

ネットワークの各層で何が起きるか: - 入力層: x そのもの - 隠れ層 1: $a_1(x) = f(W_1 x)$ (活性化関数を通った値) - 隠れ層 2: $a_2(x) = f(W_2 a_1(x))$ - $\cdots$ - 出力層: $y(x) = f(W_L \cdots)$

発想: これらの中間計算から、「**入力 x 専用のクィバー表現 W_f^x**」を作れます。作り方は、各矢印 (重み) について:

- 元の重み W_α を、**活性化関数の「瞬間的な傾き」で補正した値**に置き換える

「瞬間的な傾き」とは、終点ニューロンでの「活性化後の値 $a_j(x)$ を活性化前の値 $p_j(x)$ で割ったもの」:

$$(W_f^x)_\alpha = \frac{a_j(x)}{p_j(x)} \cdot W_\alpha$$

何が起きているか: - 非線形な活性化関数 (ReLU 等) を、「重みのスケール変換」として吸収した - 結果として W_f^x は、非線形関数を含まない、純粋な **線形なクィバー表現**になる - しかも W_f^x と、もとの x に対するネットワークの出力 $F(x)$ は、**1 対 1 で対応する**

決定的な定理 (論文の定理 6.4):

$$W_f^x \text{ の情報だけから、入力 } x \text{ に対するネットワークの出力 } F(x) \text{ が完全に再構成できる}$$

つまり、**(入力 x, ネットワーク)** のペアは、**1 つのクィバー表現 W_f^x に圧縮**できる。これが主張 2 の核心です。

8.4 主張 3 の詳細 — モジュライ空間上のデータ雲

W_f^x は、5 章で述べた「同型」の自由度を持っています。ニューロンの並べ替え等で同型な別の表現に変換できる。

「同型な表現を 1 つの点にまとめる」と、入力 x はモジュライ空間 $\mathcal{M}$ の 1 点 $\Phi(x)$ になります:

$$\Phi : \text{Input space} \to \mathcal{M}, \quad x \mapsto [W_f^x]$$

(ここで $[\cdot]$ は同型類を表す記号)

入力データ集合 $\{x_1, x_2, \ldots, x_N\}$ が与えられたとき、対応する $\Phi(x_i)$ 達は、モジュライ空間上の **点の雲**になります。

これが manifold hypothesis の実装: データ科学で経験的に言われていた「高次元データは低次元多様体に乗る」という仮説が、**モジュライ空間上の点雲**として具体的に構成されたのです。

8.5 主張 4 の詳細 — 容量との一致

「容量 (capacity)」とは、ニューラルネットがどれだけ複雑な関数を表現できるかの尺度です。

Zheng ら (2019) は、ReLU ネットワークに対し **basis-path norm** という独自の方法で容量を計算しました。これは純粋に統計学・機械学習理論の動機から設計されたものです。

Armenta & Jodoin (2021) は、全く別の代数幾何学的動機から **モジュライ空間の次元**を計算しました。

結果: 両者が **同じ数**になった。

これは驚くべきことです。2 つの全く違う動機から出てきた数が一致する。これは「背後に同じ深い構造がある」ことを強く示唆します。

8.6 論文の構成・読み方

本論文を読みたい人のためのメモです。

- 論文は 7 章構成で、全 50 ページほど。
- 1-3 章は導入と動機
- 4 章: network quiver の定義と基本性質
- 5 章: 活性化関数の数学的定義
- 6 章: データ表現 W_f^x の定義と定理 6.4
- 7 章: 9 つの帰結 (pruning、活性化関数の学習、汎化等への含意)

読むのが難しい場合: 4 章と 6 章だけ読めば主題は掴めます。5 章と 7 章はより技術的です。

8.7 この論文の影響

Armenta & Jodoin (2021) は、この分野の後続研究群を誘発しました (8 章で 6 本を紹介します)。なお、本稿執筆時点での正確な被引用数は Google Scholar などで直接ご確認ください。

---

9 その後の発展 — 6 つの論文のダイジェスト

この章の役割: Armenta & Jodoin (2021) の後、同じ枠組みを使って何が研究されたかを 6 本の論文で概観する。各論文について「何を目指したか」「何が面白いか」「限界」の 3 点に絞る。

読み方: 細部は飛ばし、「各研究は何を目指しているか」だけ掴んでください。気に入ったものを後で深掘りしてください。

9.1 論文 1: モジュライ空間を行列として書く (2022)

著者: Armenta, Brüstle, Hassoun, Reineke
掲載: Linear Algebra and Its Applications 650, 98-131 (2022)
一言で: モジュライ空間の各点を、具体的な行列として計算できるようにした

詳細:

- Armenta & Jodoin (2021) ではモジュライ空間は「抽象的な空間」だった
- この論文では、**二重框付け (double framing)** という技法で、モジュライ空間の各点を **具体的な行列**として表現可能にした
- 共著者の **Markus Reineke** は、クィバーのモジュライ空間に関する論文を長年にわたり発表してきた数学者です。特に framed quiver moduli を扱った論文 (arXiv:math/0411101, 2004) や survey paper (arXiv:0802.2147, 2008) が有名で、2000 年代以降この分野を牽引してきました。彼がこの AI 応用分野に参入したことで、数学的信頼性が大きく増した

何が面白いか:

- 「データ x → モジュライ空間の点」が、さらに「→ 具体的な行列」として計算可能になった
- これは後の論文 3 (OOD 検出等) の基礎

限界:

- 計算コストが高く、大規模モデルには未適用

9.2 論文 2: ニューラル・テレポーテーション (2023)

著者: Armenta ら 8 名
掲載: Mathematics 2023, 11(2), 480
一言で: 同じ関数のままパラメータだけを移動させる「テレポーテーション」を実験で検証した

詳細:

- ニューラルネットでは、重みを特定の仕方で変えても、表現する関数は変わらないことがある (5 章で扱った「同型」)
- 著者らはこれを **テレポーテーション**と呼び、**実験的に効果を検証**した
- 結果:
  - 関数は変わらないが、**損失地形の局所的な形が変わる** (sharpness が変化)
  - **勾配の大きさを人為的に増やせる** (勾配消失問題への応用可能)
  - **損失が最小になる地点を鋭くできる** (汎化性能への影響可能)

何が面白いか:

- 抽象的だったモジュライ空間の理論が、**実用的な訓練テクニックに結びついた**最初の例
- AI 訓練の新しい武器の候補

限界:

- 実験は MNIST・CIFAR 規模の小〜中モデルで実施。大規模 LLM での効果は未検証
- テレポーテーションを最適化に自動組み込む方法はまだ未開発

9.3 論文 3: 解釈可能性への応用 (2024)

著者: Armenta ら
arXiv: 2409.13163
一言で: 「隠れ層の活性化値を見るだけでは AI の内部は分からない。真の情報はクィバー表現にある」と主張

詳細:

- 近年の AI 解釈可能性研究 (Anthropic や OpenAI が注力) の多くは、**隠れ層の活性化値**を見て AI の内部状態を推測する
- この論文は、**隠れ層の活性化は内部状態の一部に過ぎず、完全な情報ではない**と主張する
- 正しい完全な情報は W_f^x (データのクィバー表現) にある
- 論文 1 の方法で W_f^x を行列化し、以下の応用を実装:
  - **OOD 検出**: 訓練データと違う分布のサンプルの検出
  - **敵対的サンプル検出**: AI を騙すために加工された入力の検出

何が面白いか:

- Mechanistic Interpretability の基礎に対する根本的な問いかけ
- AI のブラックボックス化を数学的に解き明かす道

限界:

- Transformer (LLM) での実験はまだ
- 計算コストが実用化には高い

9.4 論文 4: モデルを 20-40% 圧縮する (2021)

著者: Ganev, van Laarhoven, Walters
arXiv: 2107.02550 (ICLR 2022)
一言で: **特殊な活性化関数を使う NN なら、情報損失なく 20-40% サイズを削減できる**

詳細:

- ニューラルネットは**冗長なパラメータ**を多数持つ。これを削減できれば小さく高速な NN になる
- 著者らは、活性化関数を **radial activation** (ベクトルの長さだけに依存する活性化) に限定することで、冗長性を **厳密に特定**できることを示した
- 具体的には、線形代数の QR 分解を NN に拡張した技法
- 結果:
  - 20-40% の幅削減に成功
  - 精度低下は 1% 未満
  - 圧縮後の NN を訓練することが、元の大きな NN の訓練と数学的に等価

何が面白いか:

- モデル圧縮に **理論保証**がつく数少ない研究
- 訓練前にモデルを縮小できる (従来の pruning は訓練後)

限界:

- **Radial activation** 限定。ReLU や sigmoid など通常の活性化関数では、同じ対称性は使えない
- 実用的な大規模モデルでの検証はまだ

9.5 論文 5: アーキテクチャの統一言語 (2022)

著者: Ganev, Walters
arXiv: 2207.12773
一言で: **「Quiver Neural Network」という統一概念で、複雑な NN アーキテクチャを全て書けるようにした**

詳細:

- 論文 4 を、より一般的な枠組みに拡張
- **Quiver Neural Network** という概念を導入し、残差接続、attention、複雑な分岐構造など

を全てクィバーの違いとして記述

何が面白いか:

- NN 設計の共通言語を提供
- 将来、この言語で新アーキテクチャを自動設計できる可能性

限界:

- 理論中心で、実装例はまだ少ない
- PyTorch や JAX 等のフレームワークには未実装

9.6 論文 6: Softmax は代数幾何から自然に出てくる (2022)

著者: Jeffreys, Lau
掲載: Foundations of Computational Mathematics 22, 1193-1234 (2022)
一言で: 深層学習で経験的に使われる softmax 関数が、代数幾何学から必然的に出てくる

詳細:

- 著者らは、モジュライ空間に **高度な幾何学的構造** (Kähler 計量、Hermitian 計量) を入れた
- さらに、**トーリック多様体**という代数幾何学のある種の多様体族からの **自然な写像**として、活性化関数を構成した
- **驚くべき結果**: 特殊ケース (複素射影空間) を選ぶと、出てくる活性化関数が **softmax そのもの**になった

なぜ複素射影空間から softmax が出てくるのか (直感的説明):

複素射影空間 $\mathbb{CP}^n$ は、複素数 $n+1$ 個の組 $(z_0, z_1, \ldots, z_n)$ を「全体を同じ複素数倍しても同じ点」と同一視した空間です。この空間には **モーメント写像 (moment map)** という特別な写像があり、これは各点に「長さの二乗の比率」

$$\left(\frac{|z_0|^2}{\sum_i |z_i|^2}, \frac{|z_1|^2}{\sum_i |z_i|^2}, \ldots, \frac{|z_n|^2}{\sum_i |z_i|^2}\right)$$

を対応させます。各成分は $[0,1]$ で、和が 1 になります (確率分布!)。

ここで softmax の正体が見える: softmax 関数 $\mathrm{softmax}(x_i) = e^{x_i}/\sum_j e^{x_j}$ は、入力を指数関数で正値化し、その合計で割って確率分布にする操作。モーメント写像の「絶対値の二乗で割る」と構造的に同じです (指数関数による正値化 $\boxtimes$ 絶対値の二乗による正値化、どちらも「非負にしてから正規化」)。

より厳密には: Jeffreys & Lau は「複素射影空間のモーメント写像を適切な座標で表すと、入力の成分を e^{x_i} 型に変換してから正規化する操作が自然に現れる」と示しました。これは softmax と数学的に等価です。

つまり、深層学習の分類タスクで経験的に「うまくいく」とされてきた softmax は、「**確率分布の空間を複素射影空間として見たときの、最も自然な写像**」だったのです。

何が面白いか:

- 経験則で使われていた softmax が、**代数幾何学的必然性を持つ関数**であることの発見
- 他のトーリック多様体 (グラスマン多様体、フラグ多様体) から **新しい活性化関数**を構成する道が開けた
- 分類タスクの出力層に「なぜ softmax か」という問いに、数学的に決定的な答えを提供

限界:

- 数学的に非常に難しく、機械学習コミュニティに広く理解されていない
- 実装・実験は限定的

注: 本論文の難解な数学を完全に理解する必要はありません。「softmax は、確率分布の空間を複素射影空間として見たときの自然な写像として出てくる」という事実を押さえれば十分です。

9.7 論文 7: CNN のクィバー版 (2023)

著者: Wanditra, Muchtadi-Alamsyah, Nasution
掲載: Symmetry 15(12), 2110 (2023)
一言で: **畳み込みニューラルネット (CNN) 専用のモジュライ空間を構成した**

詳細:

- CNN (画像処理で使われる NN) の核心は畳み込み演算
- 著者らは、畳み込み演算を **巡回群 (cyclic group)** という対称性として捉えた
- 各頂点に「巡回群の群代数」を置き、矢印は「循環行列 (circulant matrix)」で表現
- 結果、**CNN 専用のモジュライ空間**が得られた

何が面白いか:

- Armenta & Jodoin の枠組みが、対称性を持つ特殊な NN にも自然に当てはまる
- 畳み込みの対称性を数学的に厳密に記述

限界:

- 数学的定式化中心で、応用実験は限定的

9.8 6 本のまとめ

論文	目標	応用先
1. Armenta et al. (2022)	モジュライ空間の精緻化	理論的基礎の強化
2. Armenta et al. (2023)	テレポーテーション	訓練アルゴリズムへの応用
3. Armenta et al. (2024)	解釈可能性	OOD 検出、敵対的サンプル検出
4. Ganev et al. (2021)	モデル圧縮	20-40% サイズ削減 (radial NN)
5. Ganev & Walters (2022)	統一言語	アーキテクチャ設計
6. Jeffreys & Lau (2022)	活性化関数の代数幾何	softmax の必然性
7. Wanditra et al. (2023)	CNN 特化	畳み込みのクィバー化

大きな流れ: - 理論的精緻化: 論文 1, 5, 6, 7 - 実用応用: 論文 2 (訓練), 3 (解釈), 4 (圧縮)

---

10 大規模言語モデルへの応用可能性

この章の役割: 2026 年現在、モジュライ空間の枠組みがテキストや LLM に直接適用されているかを正直に報告する。

10.1 Armenta & Jodoin のテキストへの明示的言及

論文中で著者らは以下のように述べています。

> "high-dimensional data (typically images **and text**) live on a thin and yet convoluted manifold"

(高次元データ (典型的には画像およびテキスト) は、薄くかつ複雑な多様体上に存在する)

したがって **著者自身はテキストへの適用を想定**しています。

10.2 現状は未開拓

2026 年 4 月現在、以下は **確認できていません**:

- GPT、Claude、BERT 等の大規模言語モデルに対する、モジュライ空間の具体的構成
- Transformer 全体をクィバー表現として解析した査読論文
- LLM の hallucination を OOD 検出の枠組みで解析した研究

つまり、**具体的な LLM への応用はまだ誰も書いていない**のが正直なところです。

10.3 理論的には何が可能か (筆者による独自考察)

注記: 本節の (a) (b) (c) は、本サーベイ論考執筆にあたり筆者 (Étale Cohomology) が独自に考察したアイデアであり、執筆時点 (2026 年 4 月) で対応する査読論文は存在しません。つまり、これらは既存研究のまとめではなく、**本サーベイ論考の独自の貢献**として提示されるものです。実証研究を始めたい読者の参考になれば幸いです。

以下は「やれそう」と推測できる方向性です。

(a) Transformer のクィバー記述 (筆者による提案):

Transformer を構成要素に分解すると: - **自己注意** (self-attention): トークン間の完全グラフ型クィバー - **MLP 層**: 既存の記述そのまま - **残差接続**: 別の矢印を追加 - **LayerNorm**: 活性化関数の一部として扱う

したがって、**Transformer 全体を network quiver として記述することは原理的に可能**と筆者は推測します。実装はまだ。

(b) LLM の多言語埋め込みの比較 (筆者による提案):

日本語と英語の LLM 埋め込み空間を、共通のモジュライ空間に射影して比較する応用が考えられます。文化固有の概念 (例: 「根回し」「義理」) がモジュライ空間のどこに配置されるかを調べることで、多言語 LLM の内部構造を解析できる可能性があります。

(c) LLM の Hallucination 検出 (筆者による提案):

論文 3 の「クィバー表現による OOD 検出」を LLM に適用し、hallucination を検出する技法が考えられます。ハルシネーションする入力が、訓練データのモジュライ空間上の点雲から外れた「奇妙な場所」に写像されるかを検証することが研究テーマになります。

10.4 まだ誰も書いていない研究テーマ

読者がもし研究を始めたいなら、以下のテーマがまだ未開拓です。

- 小規模 Transformer のクィバー表現の具体的構成
- Attention 機構の代数的解析
- LLM の hallucination とモジュライ空間の関係
- BERT/GPT 系の解釈可能性を W_f^x で解析

本サーベイ論考の執筆時点で、これらを直接扱った査読論文は見つかっていません。

11　現状と未解決問題

11.1　現時点で確立されたこと

1. ニューラルネットはクィバー表現として厳密に書ける (Armenta & Jodoin 2021)
2. データはモジュライ空間上の点として写像される (同上)
3. モジュライ空間の次元 = NN の容量 (独立の 2 つの量の一致)
4. 対称性を使ったモデル圧縮が可能 (Ganev ら 2021, 2022)
5. softmax が代数幾何学的必然性を持つ (Jeffreys & Lau 2022)
6. 神経表現の解釈にモジュライ空間が使える (Armenta ら 2024)

11.2　未解決の理論問題

1. **一般の活性化関数での圧縮**: ReLU、sigmoid など通常の NN での同様の理論保証付き圧縮アルゴリズムはまだない
2. **モジュライ空間の幾何的性質**: 連結性、特異点の分類等の基本的事項が未解明
3. **Manifold hypothesis の実データでの検証**: 実データが本当にモジュライ空間の「薄い部分集合」に集中するかの実証研究が不足
4. **学習ダイナミクスの理論**: 勾配降下がモジュライ空間上でどう動くかの完全理論はまだ
5. **生成モデルへの拡張**: VAE、Diffusion model はまだ

11.3　未解決の工学問題

1. **Transformer / LLM への実装検証**: 今後の研究課題
2. **大規模計算の効率化**: モジュライ空間の次元が億単位の場合の計算
3. **既存フレームワークへの統合**: PyTorch 等への組み込みはまだ

11.4　本分野の全体評価

強み:

- 数学的厳密性 (近似でなく 1:1 対応)
- 数学と統計の独立した量の一致 (主張 4) による深い示唆
- 複数の実用応用が既に派生している

弱み:

- 大規模モデルでの実験例が少ない
- 機械学習コミュニティへの浸透不足 (数学難易度が高い)
- 実装ツール化が追いついていない

今後の展望 (筆者見解):

- 小中規模 Transformer での実装検証が数年以内に出る可能性
- 解釈可能性研究で再評価される可能性
- 2030 年頃までに、モデル圧縮・解釈可能性のどちらかで実用ツールが登場する可能性

Appendix A. 主要論文の年表

年	著者	タイトル (略)	分類
1972	Gabriel	Unzerlegbare Darstellungen	クィバー表現論の基礎
1994	Nakajima	Instantons on ALE spaces	数学の基礎
1994	King	Moduli of representations	モジュライ空間の基礎
2008	Reineke	Framed quiver moduli	Framed 版の基礎
2020	Armenta & Jodoin (v1)	Representation Theory of NN	本系譜の起点
2021	Armenta & Jodoin	(改訂・Mathematics 掲載)	同上
2021	Jeffreys & Lau	Kähler Geometry	代数幾何版
2021	Ganev ら	QR decomposition for radial NN	モデル圧縮
2022	Armenta ら	Double framed moduli spaces	精緻化
2022	Ganev & Walters	Quiver Neural Networks	一般化
2023	Armenta ら	Neural Teleportation	訓練
2023	Wanditra ら	CNN quiver with cyclic groups	CNN 特化
2024	Armenta ら	Hidden Activations Are Not Enough	解釈可能性

Appendix B. 用語集

用語	やさしい言い換え	詳細
クィバー	矢印つきのグラフ	同じ 2 点を結ぶ複数の矢印やループも OK
ベクトル空間	数字の組を全部集めた袋	$\mathbb{R}^n$ (n 次元の実ベクトルの袋)
線形写像	行列 (有限次元の場合)	ベクトル空間間の変換ルール
クィバー表現	各点に「数字の袋」、各矢印に「行列」を置いた図	クィバーの各頂点にベクトル空間、各矢印に線形写像
同型	座標を取り替えたら一致する関係	基底変換で一致する表現は「同じ」と見做す
同型類	互いに同型な表現を 1 つにまとめたもの	代表者を決めて同値類で分類
モジュライ空間	同型類を点とする地図	表現空間を基底変換同値で割った商空間
Network quiver	ニューラルネットを書くクィバー	入力/隠れ/出力/バイアスのニューロンを頂点とする
活性化関数	ReLU, sigmoid 等の非線形関数	各ニューロンで適用される 1 変数非線形関数
容量 (capacity)	NN の表現能力の尺度	VC 次元や path norm 等で測る統計的量

用語	やさしい言い換え	詳細
Manifold hypothesis	高次元データは低次元の曲がった面に乗る、という仮説	多様体仮説
Radial activation	ベクトルの長さだけに依存する活性化	本サーベイ論考では立ち入らない。興味があれば Appendix C の Ganev et al. (2021) を参照
Softmax	分類問題の出力層でよく使う関数	実数ベクトルを確率分布に変換
OOD	訓練データと違う分布のデータ	Out-Of-Distribution
Kähler 計量	モジュライ空間に距離を入れる精巧な構造	本サーベイ論考では立ち入らない。興味があれば Appendix C の Kirillov (2016) の教科書を参照
トーリック多様体	代数幾何学のある特殊な多様体族	本サーベイ論考では立ち入らない。興味があれば Jeffreys & Lau (2022) と Appendix C の教科書を参照
GIT	Geometric Invariant Theory	モジュライ空間を厳密に構成するための代数幾何学の道具。本サーベイ論考では立ち入らない

Appendix C. 参考文献

11.5 C.1 本系譜の直接の研究論文

- Armenta, M., & Jodoin, P.-M. (2021). The Representation Theory of Neural Networks. Mathematics, 9(24), 3216. https://doi.org/10.3390/math9243216
- Armenta, M., Brüstle, T., Hassoun, S., & Reineke, M. (2022). Double framed moduli spaces of quiver representations. Linear Algebra and Its Applications, 650, 98-131.
- Armenta, M., Judge, T., Painchaud, N., Skandarani, Y., Lemaire, C., Gibeau Sanchez, G., Spino, P., & Jodoin, P.-M. (2023). Neural Teleportation. Mathematics, 11(2), 480.
- Armenta, M. et al. (2024). Hidden Activations Are Not Enough: A General Approach to Neural Network Predictions. arXiv:2409.13163.
- Ganev, I., van Laarhoven, T., & Walters, R. (2021). Universal approximation and model compression for radial neural networks. arXiv:2107.02550.
- Ganev, I., & Walters, R. (2022). Quiver Neural Networks. arXiv:2207.12773.
- Jeffreys, G., & Lau, S.-C. (2022). Kähler Geometry of Framed Quiver Moduli and Machine Learning. Foundations of Computational Mathematics, 22, 1193-1234.
- Wanditra, L. C., Muchtadi-Alamsyah, I., & Nasution, D. (2023). Artificial Neural Networks Using Quiver Representations of Finite Cyclic Groups. Symmetry, 15(12), 2110.

11.6 C.2 数学的基礎 (必要に応じて)

- Gabriel, P. (1972). Unzerlegbare Darstellungen I. Manuscripta Mathematica, 6, 71-103.
- King, A. D. (1994). Moduli of representations of finite-dimensional algebras. Quarterly J. Math., 45, 515-530.
- Nakajima, H. (1994). Instantons on ALE spaces, quiver varieties, and Kac-Moody algebras. Duke Math. J., 76, 365-416.
- Reineke, M. (2008). Framed quiver moduli, cohomology, and quantum groups. J. Algebra, 320, 94-115.
- Assem, I., Simson, D., & Skowroński, A. (2007). Elements of the Representation Theory of Associative Algebras. Cambridge University Press.
- Kirillov, A., Jr. (2016). Quiver Representations and Quiver Varieties (Graduate Studies in Mathematics, 174). American Mathematical Society.

11.7 C.3 関連する機械学習理論

- Zheng, S., Song, Y., Leung, T., & Goodfellow, I. (2019). Capacity Control of ReLU Neural Networks by Basis-path Norm. AAAI.
- Dinh, L., Pascanu, R., Bengio, S., & Bengio, Y. (2017). Sharp Minima Can Generalize For Deep Nets. ICML.

License

本サーベイ論考は CC BY 4.0 のもとで公開される。引用時は以下を推奨:


```
@misc{etale2026moduli_nn,
  author = {Étale Cohomology},
  title  = {ニューラルネットワークとモジュライ空間: クィバー表現論でAIを理解するサーベイ論考},
  year   = {2026},
  note   = {Technical Survey for undergraduate level readers}
}
```

Issues, PRs welcome. 分かりにくい箇所の報告を歓迎します。